湖南科技大学计算机科学与工程学院

智能终端系统及应用开发课程设计报告

专业班级： 24 级物联网工程三班

姓　名： 李思民

学　号： 2405040303

指导教师： 黄晶

时　间： 2026.10.5~2026.10.26

地　点： 逸夫楼 433

指导教师评语：

成绩：　　**等级：**

签名：

目录

一　实验题目.....3
二　实验目的.....3
三　总体设计.....4
3.1 需求分析与功能边界.....4
3.2 技术方案与模块划分.....4
3.3 系统架构与数据流.....4
四　详细设计.....5
4.1 主界面导航与组件路由.....5
4.2 首页浏览与内容发现.....6
4.3 网络请求与首页分页.....7
4.4 视频播放与生命周期管理.....9
4.5 用户登录与互动状态.....11
4.6 广场与内容分类.....12
4.7 本地浏览记录与数据持久化.....13
五　实验结果与分析.....15
5.1 功能验证与测试设计.....15
5.2 运行现象与结果分析.....16
5.3 问题分析与改进方向.....18
六　小结与心得体会.....19

一　实验题目

本项目基于 Android 的组件化短视频应用设计与实现。项目工程名称为 shortVideo，围绕视频内容浏览与播放，组织首页、广场、发现和个人中心四个主要入口，形成“用户登录—浏览内容—进入播放—互动收藏与历史回看”的使用流程。

客户端采用 Java 与 Kotlin 混合开发，使用 MVVM、DataBinding 和组件路由组织页面，通过 Retrofit 与 OkHttp 对接既有远程接口，并利用 Room 保存本地浏览记录。设计重点为 Android 客户端的功能组织、数据交互和媒体资源管理，服务端接口由现有服务提供。

本项目源代码已上传至 GitHub，项目仓库地址：
https://github.com/litianyan52/shortVideoRepository

二　实验目的

1. 掌握 Android 多页面应用的组织方法，将 Activity、Fragment、ViewPager2 和 RecyclerView 综合应用于内容浏览场景，理解页面切换、列表复用和生命周期之间的关系。

2. 理解 MVVM 中界面、状态与数据访问的职责，使用 LiveData 与 DataBinding 传递数据，减少界面代码与网络逻辑的直接耦合。

3. 掌握组件化工程的基本结构，利用 ARouter 完成跨模块页面跳转和参数传递，认识公共基础模块在复用和统一交互中的作用。

4. 掌握视频播放的基础流程，使用 Media3 ExoPlayer 加载媒体地址，并根据页面生命周期暂停与释放资源，避免后台播放及资源泄漏。

5. 掌握远程数据与本地持久化的协同方式，理解分页、登录令牌、浏览记录、异步数据库操作及错误状态处理。

6. 掌握功能测试的设计方法，围绕页面导航、内容加载、媒体播放、用户互动和本地存储组织测试步骤，分析运行结果与优化方向。

7. 本设计以“内容获取、界面展示、媒体播放、用户互动和数据保存”为学习主线，在实现功能的同时关注模块边界、异步操作与状态一致性。

三　总体设计

3.1 需求分析与功能边界

应用面向通过移动终端浏览短视频的用户，提供推荐、日报、分类和视频详情浏览功能。登录用户可进行点赞、收藏、评论和个人资料管理。首页采用纵向视频卡片列表，点击卡片进入独立播放页；个人资料编辑通过相册选择头像。

核心使用场景分为三类：游客浏览内容并播放视频；登录用户进行收藏、点赞和评论；用户通过收藏与本地记录回到已关注的内容。用户操作应能够获得明确反馈，网络不可用时保留必要的返回与重试入口。

3.2 技术方案与模块划分

工程以 app 为集成入口，业务模块依赖公共基础能力，通过路由访问其他业务页面。根工程 isModule=false 时，各业务模块以库的形式参与集成；切换组件模式时，公共构建脚本调整插件与清单文件，以支持模块调试。

系统各模块的划分及主要职责见表 3-1。

表 3-1　系统模块划分与主要职责

模块	主要职责
app	应用启动、底部四入口、Fragment 切换
library_base	页面基类、路由常量、用户管理、Room、工具类
network 与 video_data	Retrofit 请求封装、统一响应和业务数据模型
feature_home	推荐与日报容器，复用视频列表
feature_mediaplay	播放、互动、搜索、分类列表、收藏、浏览记录
feature_piazza 与 feature_find	广场图文、图片浏览、分类、主题与话题入口
feature_user 与 feature_camera	验证码登录、资料设置、头像拍照

工程编译配置为 compileSdk 35、targetSdk 35、minSdk 24，Java 编译目标为 11。核心依赖包括 Retrofit 2.11.0、OkHttp 4.12.0、Glide 4.16.0、Room 2.6.1、ARouter 1.5.2 和 Media3 1.0.0。

3.3 系统架构与数据流

系统总体架构如图 3-1 所示。业务模块通过路由协作，公共层集中提供基础能力；远程接口与本地数据库承担不同的数据职责。

远程接口提供视频内容、用户资料和互动数据。用户管理器使用 EncryptedSharedPreferences 保存令牌与用户资料，Room 保存本地浏览记录。播放器通过媒体地址在线加载视频，浏览记录保存视频编号、封面、标题和访问时间，支持历史回看。

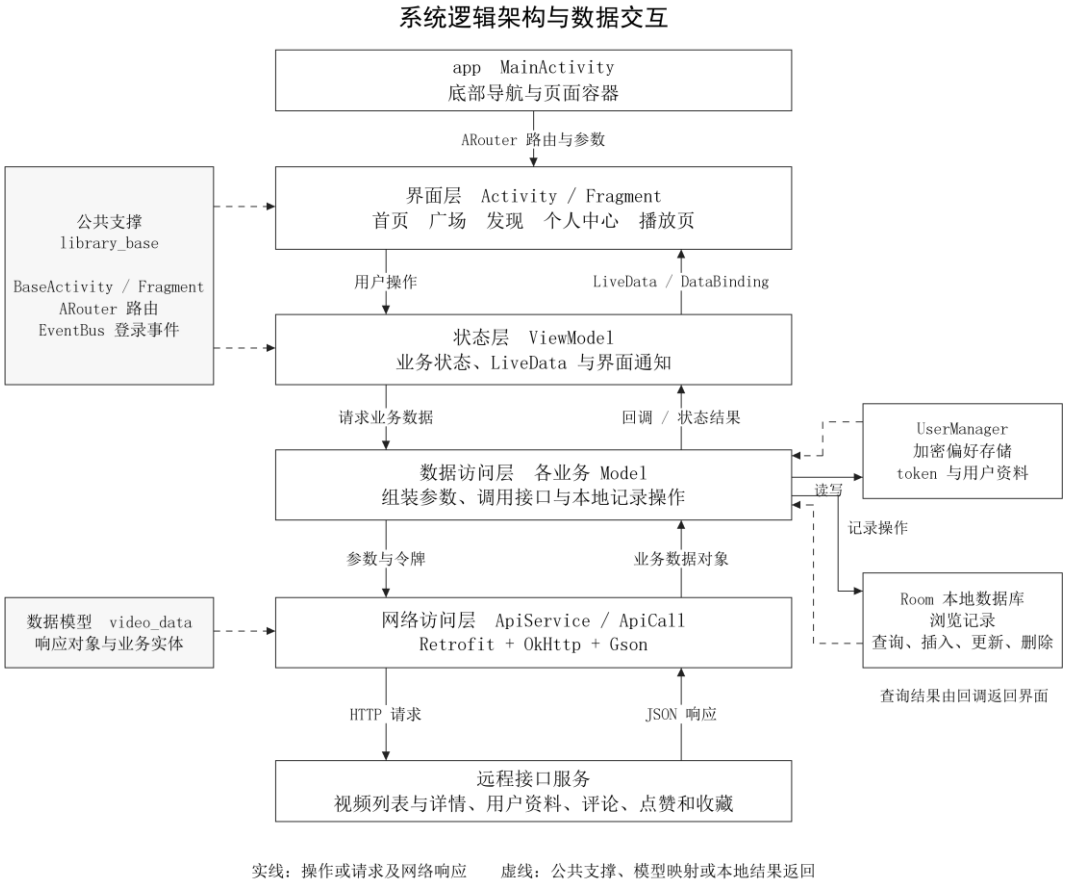


图 3-1 系统总体架构

主要数据流为：页面发起操作，ViewModel 组织状态，Model 请求 API，再将结果转换为 LiveData 更新界面。登录后广播登录事件，相关页面重新获取资料或互动状态，使个人中心与播放页保持一致。

视频列表对象 ResVideo 提供卡片展示字段，ArchivesInfo 提供播放地址、正文及互动状态，ResVideoAllInfo 聚合详情相关数据。列表项只传递编号，播放页再读取完整详情，可避免把大对象随页面跳转反复传递。

四 详细设计

4.1 主界面导航与组件路由

MainActivity 将首页、广场、发现和我的入口映射到路由路径。首次切换时创建并添加对应 Fragment，后续通过 show 和 hide 切换，保留列表位置与页面数据，支持四个入口之间的连续浏览。

路由常量集中定义在 ArouterPath 中，视频列表通过视频编号跳转播放页，目标页面通过参数注入获得编号，再请求完整详情。调用方不必直接引用目标 Activity 的具体类型，从而降低模块之间的耦合。

代码 4-1　视频列表跳转播放页的调用方式

```
ARouter.getInstance()
    .build(ArouterPath.Video.ACTIVITY_MEDIA_PLAY)
    .withInt(ArouterPath.Video.KEY_VIDEO_ID, videoId)
    .navigation();
```

代码来源：VideoListFragment.java 的路由调用方式，参数变量作简化。

路径常量确定目标页面，KEY_VIDEO_ID 统一视频编号的参数名称。播放页接收编号后调用详情接口，完成“列表选择—路由跳转—详情加载”的页面衔接。

工程公共基类承担 ViewModel 初始化、DataBinding 绑定和参数注入，并统一观察加载、提示及结束页面等状态，使业务页面集中处理自身逻辑。

底部导航通过 add、show 和 hide 复用 Fragment，减少重复创建页面和加载数据。播放页结合生命周期管理媒体资源，将页面切换与播放器控制分开组织。

4.2 首页浏览与内容发现

首页通过“推荐”和“日报”切换视频列表，发现页通过分类、主题和话题组织内容，分别支持连续浏览与定向查找。首页与发现界面如图 4-1 所示。

首页卡片包含封面、标题和作者信息。封面使用 Glide 异步加载，列表使用 RecyclerView 回收条目视图；点击卡片进入播放页。发现页的分类与主题入口将浏览范围缩小到相应内容集合。

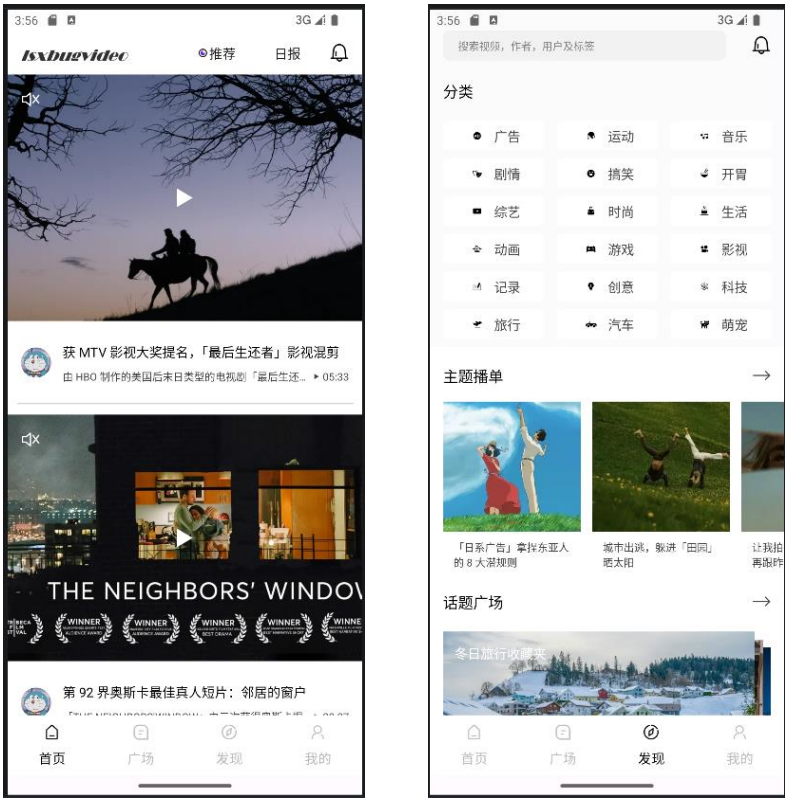


图 4-1　首页与发现界面

（a）首页推荐列表　　（b）发现分类与主题

4.3 网络请求与首页分页

网络层采用 Retrofit 定义接口、OkHttp 执行请求、Gson 解析 JSON。Java 回调封装与 Kotlin 协程封装目前并存，播放页采用 Kotlin 链路。统一响应包含 code、msg、time 和 data，列表数据进一步包含 count 与 list。

首页由 ViewPager2 承载推荐和日报两页，列表复用 VideoListFragment。首次加载与下拉刷新从第一页请求，上拉加载请求后续页；刷新替换旧数据，加载更多追加数据。每页请求十条记录，已加载条数达到服务端返回的 count 时关闭继续加载。

分页请求与列表更新流程如图 4-2 所示。

分页请求将页码与每页条数作为接口参数，由 Model 获取数据、ViewModel 更新列表状态。后续优化采用“成功后提交页码”的方案，并对同一列表的加载请求进行串行控制。

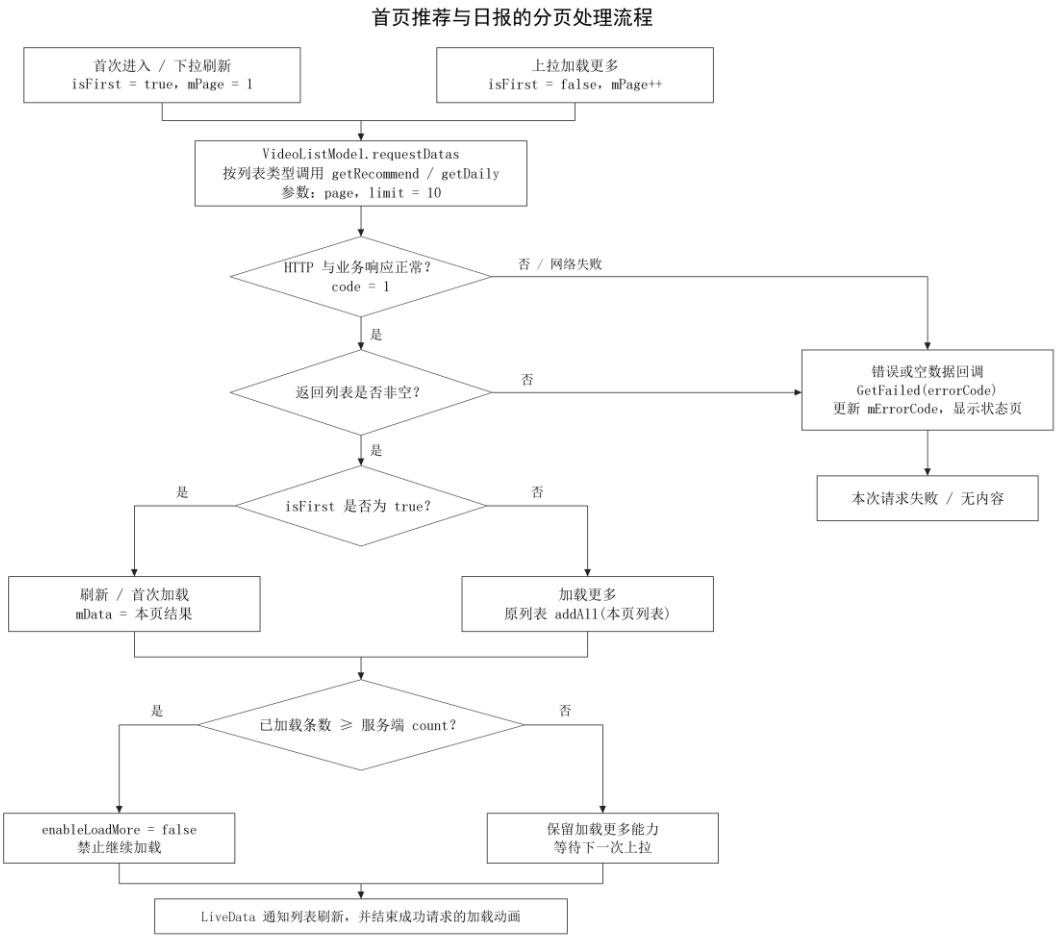


图 4-2　分页请求与列表更新流程

4.3.1 请求状态的统一处理

代码 4-2　统一封装对业务结果的判断

```
val response = call()
if (response.getCode() == successCode) {
    val data = dataType(response)
    when {
        data != null -> NetworkStatusResult.Success(data)
        else -> NetworkStatusResult.Empty(response.getMsg())
    }
} else {
    NetworkStatusResult.Error(response.getMsg())
}
```

代码来源：network/.../ApiCallKotlinVersion.kt。

封装通过 successCode 判断业务结果，再利用 dataType 提取具体数据，将返回状态归纳为 Success、Empty 和 Error。页面依据状态完成数据展示、空内容提示和错误反馈，实现网络处理与界面展示的分离。

该封装对 CancellationException 原样抛出，使协程取消继续传播，其余异常转换为网络错误结果。ViewModel 使用 viewModelScope 发起请求，随生命周期结束取消协程；播放器由独立的媒体管理类负责暂停和释放。

不同请求状态对应的界面处理设计见表 4-1。

表 4-1　网络请求状态与界面处理

场景	界面处理设计
成功返回列表	刷新替换或加载追加；检查是否重复
空列表	区分首次无内容和已经到达末页
网络或业务失败	结束加载状态，保留已有内容并提示原因
登录令牌失效	清理或更新会话，引导重新登录

4.4 视频播放与生命周期管理

实际路由指向 MediaPlayActivityKotlinVersion。播放页接收视频编号后，由 ViewModel 请求详情，Activity 观察视频数据并将 video_file 交给播放器，同时刷新评论。简介与评论由 ViewPager2 组织，使媒体区域和信息区域在同一页面中协同展示。

MediaPlayerManager 对 ExoPlayer 进行单例封装，使用应用级 Context 创建实例，并绑定到 PlayerView。切换媒体时设置 MediaItem，完成准备后播放。页面销毁时停止播放、清空媒体并释放播放器，同时清除单例引用。

代码 4-3　媒体地址装载与播放

```
public void play(String url) {
    if (mPlayer != null) {
        MediaItem mediaItem = MediaItem.fromUri(url);
        mPlayer.setMediaItem(mediaItem);
        mPlayer.prepare();
        mPlayer.play();
    }
}
```

代码来源：feature_mediaplay/.../player/MediaPlayerManager.java。

MediaItem 描述媒体资源，setMediaItem 设置播放对象，prepare 执行媒体准备，play 发起播放。四个步骤构成从媒体地址到播放器输出的调用流程。

播放页在 onStart 中允许播放，在 onPause 和 onStop 中暂停，在 onDestroy 中停止并释放播放器，使媒体资源的使用与页面生命周期对应。

播放页采用 singleTask。再次点击视频卡片时，通过 onNewIntent 接收新编号并重新请求详情，复用现有页面，完成播放内容切换。

4.4.1 播放链路与资源释放

视频播放与浏览记录的数据流程如图 4-3 所示。

onNewIntent 接收新视频编号后重新请求详情，并恢复评论加载标记。详情更新触发播放器加载媒体、界面展示视频信息以及浏览记录写入，形成播放与本地存储协同的数据流程。

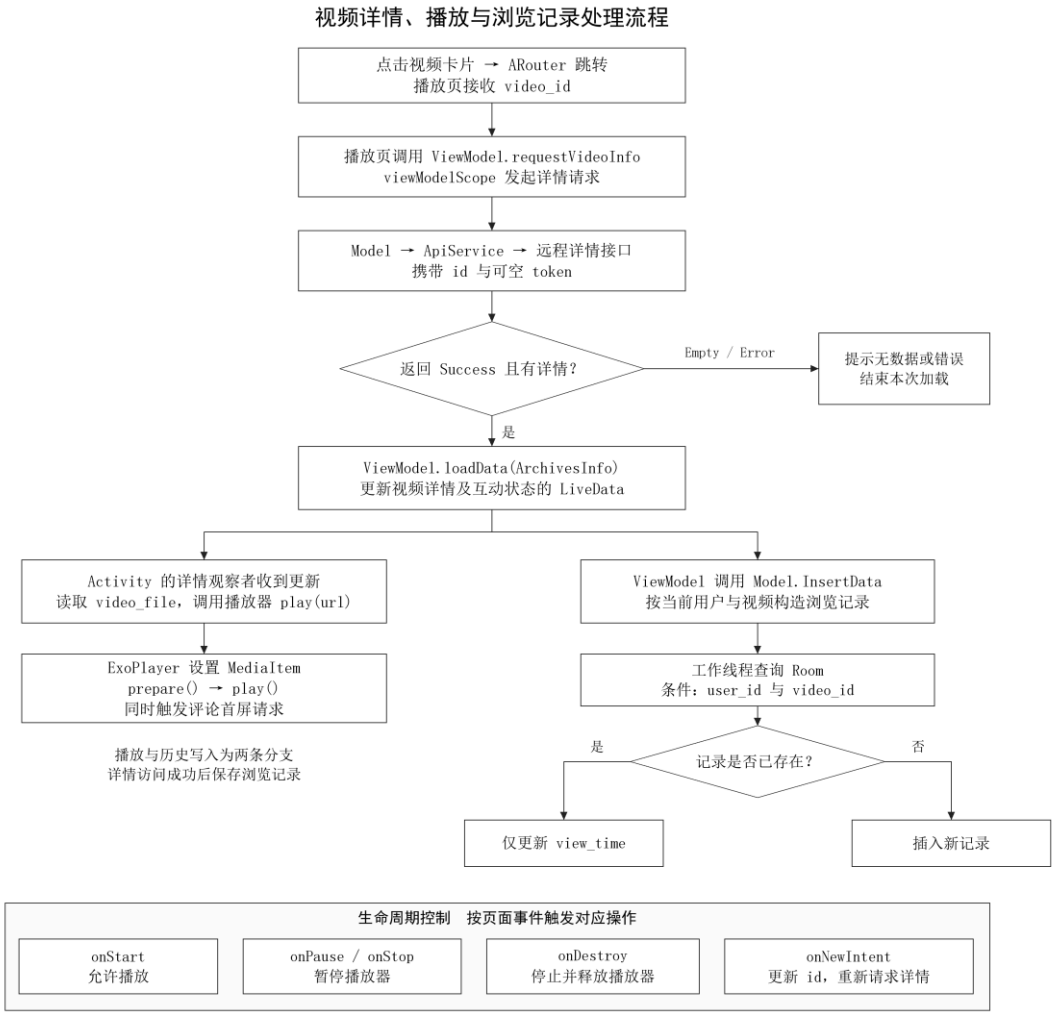


图 4-3 视频播放与浏览记录的数据流程

播放器在各生命周期入口中的处理及对应场景见表 4-2。

表 4-2 播放器生命周期处理与对应场景

生命周期入口	当前处理	对应场景
onStart	允许播放，注册事件	页面首次显示与后台返回
onPause / onStop	暂停播放	短暂遮挡与切后台
onDestroy	停止、清空媒体并释放	退出后无音频与资源残留

4.4.2 视频简介与评论界面

视频播放页的简介与评论界面如图 4-4 所示。

两张截图中的播放位置与视频内容一致，下部区域分别显示简介和评论。简介提供标题、描述、互动入口及相关内容，评论区域显示评论列表和输入入口。页面切换以同一视频详情为上下文，避免用户从简介切到评论时改变播放对象。

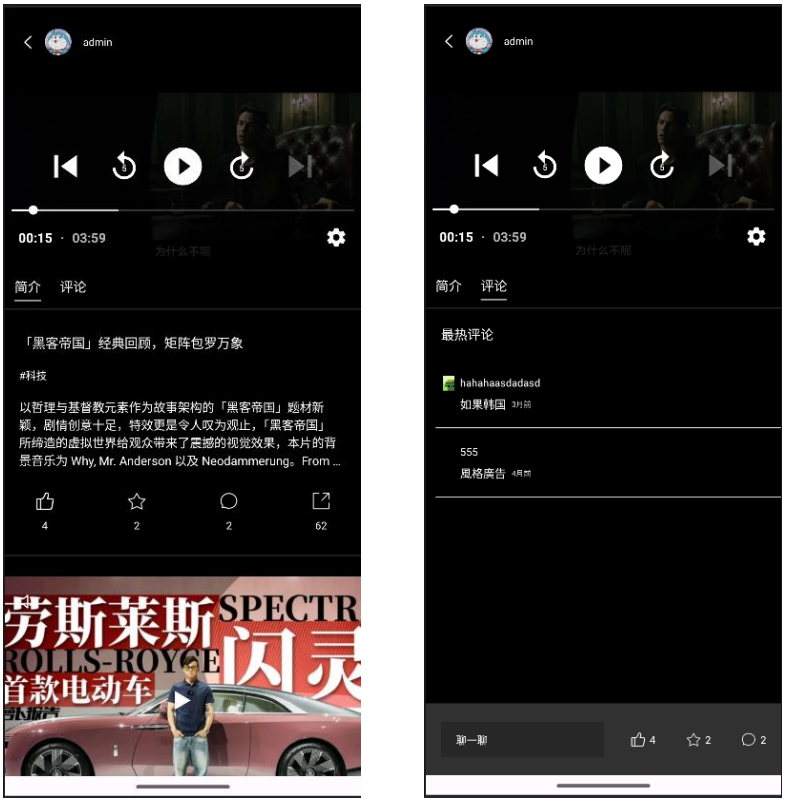


图 4-4　视频播放页的简介与评论

（a）视频简介及互动入口　　（b）同一视频的评论列表

播放页采用上部媒体播放、下部详情展示的布局。简介与评论通过标签切换，共享当前视频上下文，用户可在观看过程中查看内容介绍和评论列表。

4.5 用户登录与互动状态

用户模块提供手机号验证码登录入口。登录成功后保存令牌与资料，后续受保护接口通过 token 请求头携带身份信息。退出登录清理本地会话，并通过 EventBus 通知相关页面刷新，完成会话建立、使用和退出的管理流程。

代码 4-4　视频详情接口允许游客令牌为空

```
@GET(NetAddress.VIDEO_INFO_ADDRESS)
suspend fun getVideoInfo(
    @Query("id") id: String,
    @Header("token") token: String?
): ResBase<ResVideoAllInfo>
```

代码来源：feature_mediaplay/.../apiKotlinVersion/MediaApiService.kt。

视频详情接口将 token 定义为可空参数，支持游客浏览公开内容。点赞、收藏和评论等互动操作在 ViewModel 中检查登录状态，并使用登录令牌标识当前用户。

Kotlin 点赞与取消点赞逻辑在请求返回 Success 后更新 mIsLike、mLikes 和详情对象，再通过 LiveData 将互动状态传递给界面。

评论模块围绕当前视频组织评论列表、输入入口和删除操作。删除逻辑关联评论标识与接口请求，后续优化将采用服务端确认后更新列表的方式统一交互状态。

4.5.1 个人中心与头像选择

个人中心与系统相册选择头像的界面如图 4-5 所示。

个人中心展示昵称或脱敏账号信息，并提供收藏、浏览记录及退出入口。资料编辑调用系统照片选择器，用户通过相册选择头像图片。

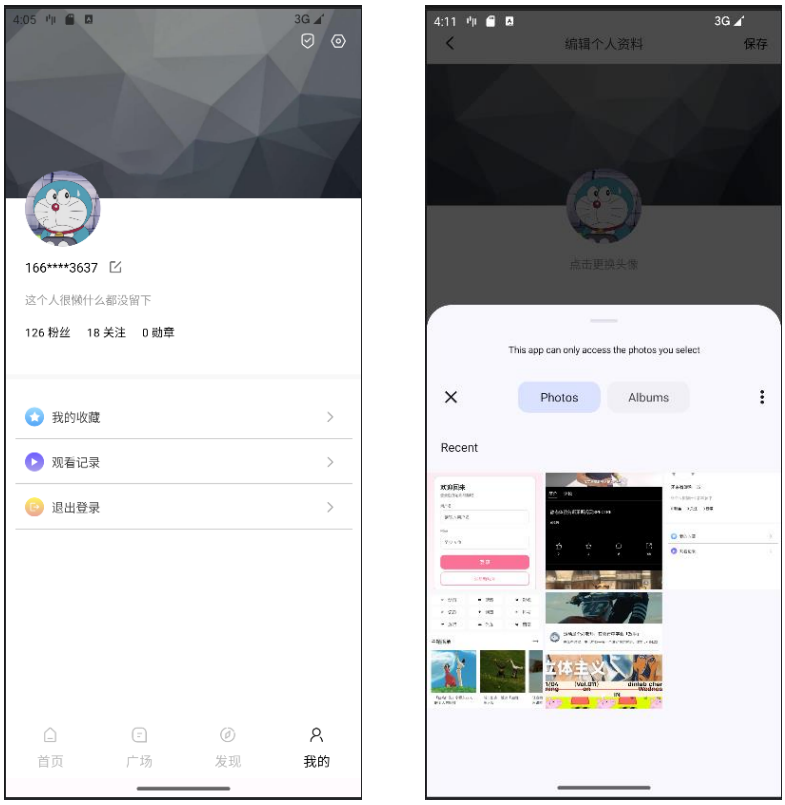


图 4-5 用户资料与头像选择界面

（a）已登录个人中心 （b）系统相册选择头像

头像选择以系统照片选择器为交互入口，将图片选择操作与个人资料编辑衔接，统一组织昵称、头像和账号信息的维护入口。

4.6 广场与内容分类

广场将轮播内容与双列图片卡片组合展示，轮播占据完整行，其余卡片按双列排列。该设计适合突出精选内容，并保留进入分类或大图浏览的入口。

广场的轮播与双列图片布局如图 4-6 所示。

适配器使用不同条目类型处理轮播与普通图片。由于第一个位置被轮播占用，普通图片的数据索引需要减去相应偏移；点击事件同样需要使用转换后的索引。图片浏览页用横向 ViewPager2 展示组图，并通过页码提示当前查看位置。



图 4-6　广场界面

广场轮播与图片网格

发现页的分类和主题播单与广场共同构成多入口浏览结构。分类入口按内容类型组织视频，主题入口聚合相关内容，广场通过轮播与图片卡片提供直观的内容导航。

4.7 本地浏览记录与数据持久化

Room 实体 UserLookRecord 对应 user_look_record 表。每次访问视频详情后，按用户编号和视频编号查找记录；已存在时更新时间，不存在时插入。读取时按浏览时间倒序排列，支持对选中视频批量删除。数据库访问在工作线程执行，结果通过主线程 Handler 返回界面。

浏览记录的主要字段、类型及作用见表 4-3。

表 4-3　浏览记录数据字段

字段	类型	作用
id	int	自增主键
user_id / video_id	String / int	区分用户及视频
cover / title / label	String	封面、标题、标签
duration	String	视频时长展示
view_time	long	浏览时间与倒序排序

代码 4-5　按用户读取并倒序展示浏览记录

```
@Query("SELECT* FROM user_look_record " +
       "WHERE user_id = :user_id " +
       "ORDER BY view_time DESC ")
List<UserLookRecord> getRecords(String user_id);
```

代码来源：library_base/.../db/UserLookRecordDao.java；长 SQL 按行拆分展示（节选）。

user_id 用于区分账号记录，view_time 用于按最近访问时间排序。写入前通过用户编号与视频编号组合查询，已有记录更新时间，新记录执行插入，构成浏览历史的维护流程。

数据库操作在工作线程执行，回调切回主线程后更新列表。浏览记录保存封面、标题和视频编号等元数据，点击记录后根据视频编号获取详情并在线播放。

4.7.1 收藏与历史回看界面

收藏列表与浏览记录界面如图 4-7 所示。

收藏列表记录用户主动保存的内容，通过远程接口读取；浏览记录保存最近访问的视频，从本机 Room 查询。两者分别提供兴趣内容管理和最近访问回看入口。

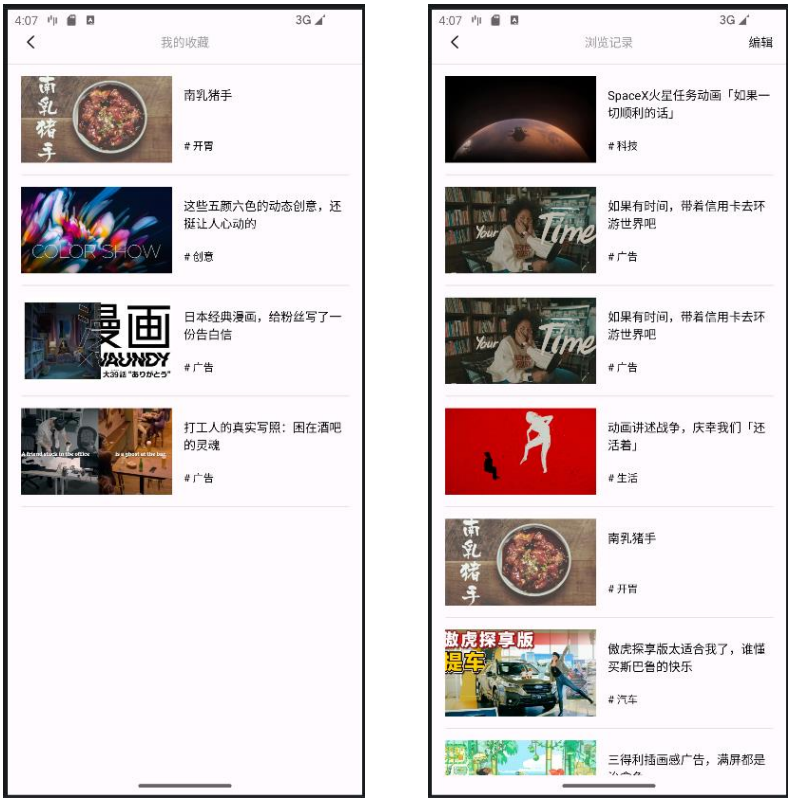


图 4-7　收藏与浏览记录

（a）远程收藏列表　　（b）本地浏览记录

浏览记录页面提供编辑入口，支持选择并删除记录；再次访问同一视频时更新时间，列表按照最近访问顺序展示。记录查询使用用户编号区分不同账号的数据。

五　实验结果与分析

5.1 功能验证与测试设计

实验围绕页面导航、视频浏览与播放、登录互动和本地记录四个方面组织验证。首页、发现、播放、评论、个人中心、广场、收藏、浏览记录和相册选择界面展示了各功能模块的交互布局。

系统功能测试步骤、预期结果及记录结果见表 5-1。

表 5-1　系统功能测试用例与结果

测试项目	测试步骤与预期结果	测试结果
四入口切换	切换后返回首页，列表与位置保持	通过
推荐与日报	刷新替换、加载追加，失败不跳页	通过
游客播放	未登录打开视频，正常读取详情	通过
视频切换	从相关列表选新视频，媒体和评论同步	通过

测试项目	测试步骤与预期结果	测试结果
生命周期	切后台、短暂遮挡及返回，暂停恢复正确	通过
登录与退出	登录后资料刷新；退出后互动被拦截	通过
点赞与收藏	新增及取消后，重进页面状态一致	通过
评论操作	提交后可见，删除失败时状态可恢复	通过
历史持久化	重复访问更新时间，删除后重进不恢复	通过
网络与权限	断网或拒绝权限，提示并可返回重试	通过

5.2 运行现象与结果分析

图 4-1 展示首页卡片与发现分类，形成内容推荐和分类查找两种浏览入口；图 4-4 展示视频简介与评论的协同布局。图 4-5 展示个人中心及系统相册选择，图 4-6 展示轮播与双列图片布局，图 4-7 展示收藏和浏览记录，构成从内容发现到播放、互动及回看的页面结构。

从架构上看，组件路由使首页可复用媒体模块中的列表和播放入口，公共列表基类减少刷新与加载逻辑的重复。MVVM 将界面状态与数据访问分开，Room 为历史回看提供本地数据基础。

5.2.1 登录前后界面对比

登录前后的界面如图 5-1 所示。登录页提供账号与验证码输入入口，登录后的设置页展示账号绑定信息和退出登录入口。

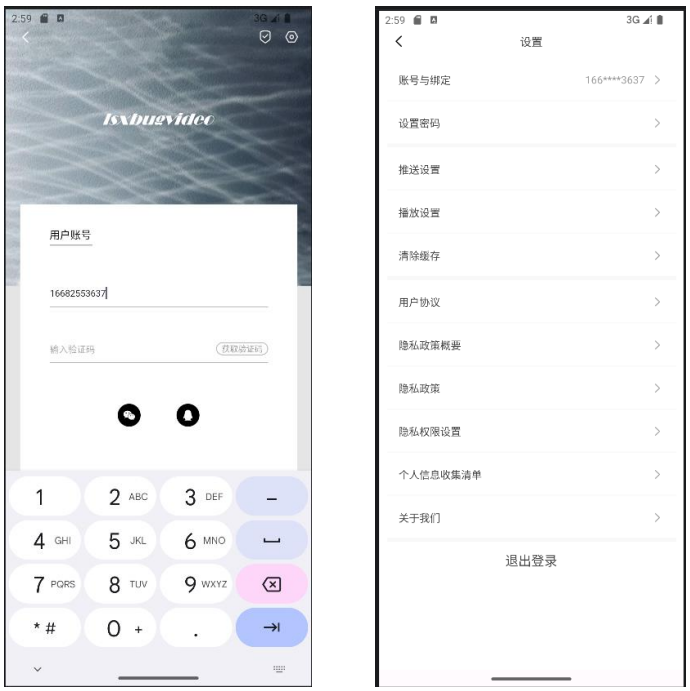


图 5-1 登录前后界面对比

（a）登录前的验证码登录页 （b）登录后的账号设置页

登录前，用户在登录页输入账号并获取验证码；登录后，设置页显示脱敏账号及“退出登录”按钮，账号管理入口随登录状态呈现相应内容。

5.2.2 点赞与收藏操作对比

同一视频在点赞与收藏操作前后的界面如图 5-2 所示，视频标题和播放位置保持一致。

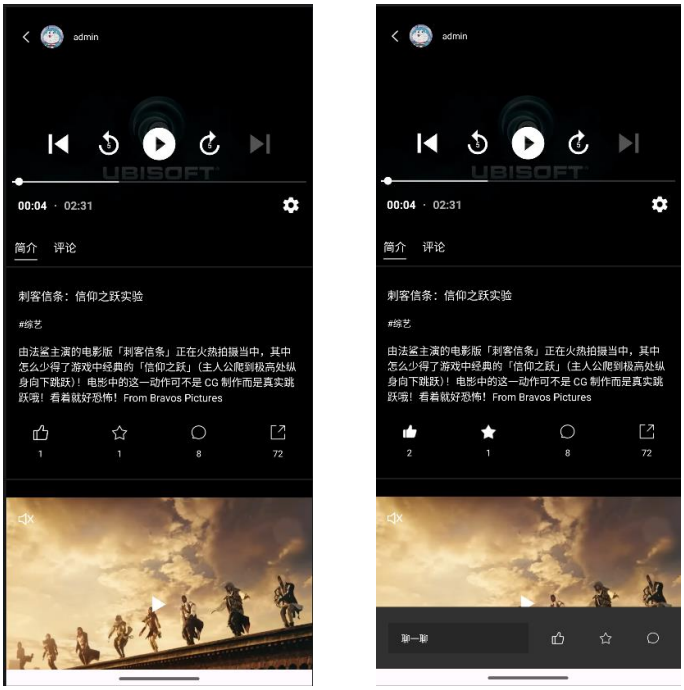


图 5-2　点赞与收藏操作前后对比

（a）操作前　　（b）操作后

操作前，点赞和收藏图标为空心；操作后，两者变为实心选中状态，点赞数量由 1 增加到 2。界面通过图标状态与点赞数值反馈用户的互动操作，视频内容和播放位置保持不变。

5.2.3 浏览记录删除前后对比

浏览记录的选择删除过程如图 5-3 所示。用户进入编辑状态，选中目标记录后完成删除。

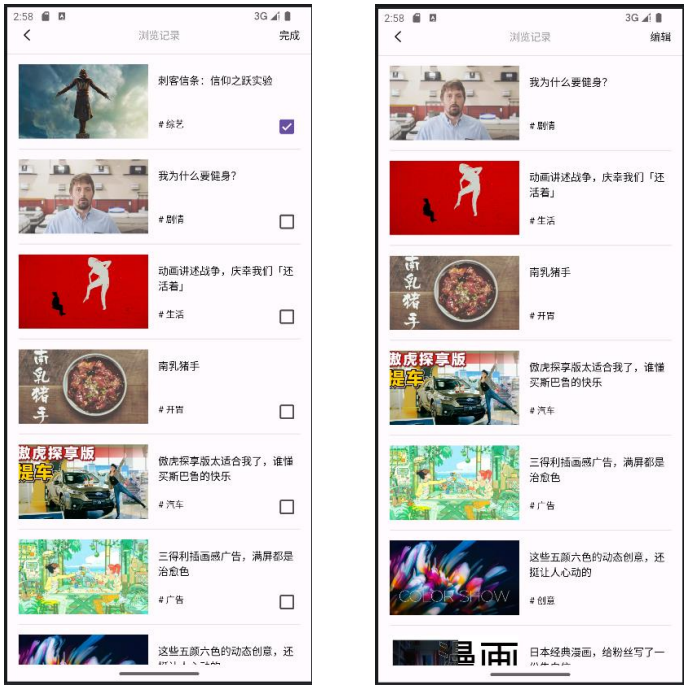


图 5-3　浏览记录删除前后对比

（a）删除前选中目标记录　　（b）删除后的记录列表

删除前，“刺客信条：信仰之跃实验”位于列表首项并处于选中状态；删除后，该记录从列表移除，“我为什么要健身？”成为首项，其余记录依次前移，页面返回普通浏览状态。

5.3 问题分析与改进方向

围绕分页、互动状态、播放控制和记录存储的进一步优化方案见表 5-2。

表 5-2　系统优化方向与实现方案

优化方向	实现方案
分页请求状态管理	成功后提交页码，失败时保留目标页码，串行处理加载请求
收藏数量计算	先获取当前数值，再执行加减并同步详情对象
评论列表更新	服务端确认后删除，统一维护列表与接口状态
播放状态管理	统一弹窗、页面及播放器的共享状态
播放恢复策略	结合页面可见状态与用户暂停状态控制恢复
浏览记录写入	增加用户与视频的组合唯一约束，使用事务组织查询与写入

5.3.1 收藏数量更新的代码分析

代码 5-1　当前收藏数量的更新表达式

```
val newCollections = mCollections.value?.toInt()
    ?: ERRORCOLLECTIONS + 1
```

代码来源：MediaPlayViewModelKotlinVersion.kt；按原表达式换行。

该表达式中，加一作用于 Elvis 运算符右侧的后备值。若 mCollections.value 能正常转换为整数，结果直接取该整数，不会执行加一；取消收藏的相应表达式也有同类问题。这会使收藏状态已变化而显示数量未变化。

代码 5-2　建议的数值更新方式

```
val current = mCollections.value?.toIntOrNull() ?: 0
val updated = current + 1
mCollections.value = updated.toString()
mArchivesInfo.value?.collection = updated
```

代码说明：收藏计数优化方案示例。

收藏计数优化方案以接口成功回调为更新入口：新增收藏执行加一，取消收藏执行减一并将下限设为零；接口返回最新计数时使用服务端值。交互层采用请求期间禁用按钮或串行处理的方式组织连续操作。

六　小结与心得体会

本课程设计围绕 Android 短视频客户端，将内容列表、媒体播放、登录互动和本地记录组织在同一工程中。通过需求梳理、模块划分和业务流程设计，我加深了对页面状态、数据访问及资源管理协同关系的理解，掌握了从功能需求到客户端实现的基本方法。

在架构方面，组件划分帮助我建立了清晰的职责意识。首页组织推荐与日报，媒体模块承担播放和互动，公共层提供路由、基类及用户管理。通过明确模块边界和公共能力的复用范围，代码结构更加清楚，也便于定位功能和维护业务流程。

在数据处理方面，我理解了网络响应、界面状态和本地存储的分工。LiveData 负责通知界面刷新，接口返回业务数据与互动结果，加密偏好存储保存会话资料，Room 维护浏览记录。这些机制共同支撑用户从内容浏览到历史回看的连续使用过程。

视频播放的实现让我进一步掌握了生命周期管理。播放器准备、暂停和释放与页面状态相对应，singleTask 配合 onNewIntent 接收新视频编号并重新加载详情，使页面复用与媒体切换形成完整的控制流程。

报告整理加深了我对技术表达的认识。通过流程图呈现数据流向，通过代码片段说明核心处理，通过界面截图展示交互布局，可以将需求、设计与实现组织为清晰的实验报告。今后的学习中，我会继续积累调试记录和测试用例，提高开发效率与文档表达能力。

后续优化将围绕分页请求、互动计数、评论更新和生命周期控制展开，进一步完善状态管理与异常反馈。同时，结合账号切换和连续操作场景组织回归测试，提升应用的交互一致性与维护效率。